Batched Dense Optical Flow in PyTorch

批量稠密光流的 PyTorch 实现与基准测试

TV-L1 & DeepFlow: OpenCV CPU vs. CUDA vs. pure-torch reimplementation

T. Long
tlong32@jhu.edu

August 2026

Middlebury pairs 480×640 · NVIDIA L40 (SLURM, ilps-cn119) · 48-core CPU login node

Motivation 研究动机

- Dense optical flow is a standard **preprocessing step at scale**: video understanding, motion features, temporal augmentation, dataset curation — millions of frame pairs, not one.
- Classic accurate methods (TV-L1, DeepFlow) are **variational**: hundreds of small iterations per pair $\Rightarrow$ slow on CPU, awkward on GPU.
- What matters for preprocessing is **batch throughput** (pairs/second over a whole dataset), not single-pair latency.
- Goal: benchmark OpenCV CPU vs. OpenCV CUDA vs. our **batched pure-PyTorch reimplementations**, on identical inputs and parameters.

中文要点　稠密光流常作为大规模视频处理的预处理步骤：要处理的是数百万帧对，而非单帧。 经典变分方法（TV-L1、DeepFlow）迭代次数多、单对速度慢；预处理场景真正重要的是 批量吞吐量。我们在相同输入与参数下对比 OpenCV CPU / OpenCV CUDA / 自研批量化 PyTorch 实现。

Benchmark Setup 实验设置

Data & hardware

- Middlebury training pairs, 480×640 grayscale (also downscaled 240×320); ground-truth flow for EPE.
- GPU: NVIDIA L40 (142 SMs), SLURM node `ilps-cn119`.
- CPU: 48-core login node.
- Env: `uv`-managed; `opencv-contrib-python-headless`, PyTorch.

Algorithm × backend matrix

	cv CPU	cv CUDA	torch
TV-L1	✓	✓*	✓
DeepFlow	✓	none	✓
Farneback	✓	✓*	—

*only with a custom CUDA build of OpenCV — pip wheels ship **without** CUDA.

中文要点　数据：Middlebury 训练集（480×640，含真值光流，用于计算端点误差 EPE）。 硬件：NVIDIA L40（142 个 SM，SLURM 节点）对比 48 核 CPU 登录节点。 注意：pip 安装的 OpenCV 不带 CUDA，需要自行编译；DeepFlow 在 OpenCV 中 没有任何 CUDA 版本。

TV-L1: Energy Formulation 能量泛函

$$E(\mathbf{u}) \;=\; \int_{\Omega} \lambda \left|\rho(\mathbf{u}(\mathbf{x}))\right| \;+\; |\nabla u_1| + |\nabla u_2| \;\, d\mathbf{x}$$

with the image residual linearized around the current warp $\mathbf{u}^0$:

$$\rho(\mathbf{u}) \;=\; I_1(\mathbf{x}+\mathbf{u}^0) \;+\; \left\langle \nabla I_1(\mathbf{x}+\mathbf{u}^0),\, \mathbf{u}-\mathbf{u}^0 \right\rangle \;-\; I_0(\mathbf{x})$$

- L^1 data term: robust to outliers/occlusions; TV term: allows sharp flow discontinuities at motion boundaries.
- Both terms non-smooth $\Rightarrow$ solved via **duality**, not gradient descent.
- Params used (OpenCV defaults): $\lambda = 0.15$, $\theta = 0.3$, $\tau = 0.25$, 5 scales (step 0.8), 5 warps, up to 10×30 primal–dual iterations.

中文要点　能量 = 全变差正则项 + 鲁棒 L^1 数据项；ρ 是在当前解 $\mathbf{u}^0$ 处线性化的亮度残差。L^1 对遮挡/离群点鲁棒，TV 项保留运动边界的不连续性。 两项都不可微，因此用对偶方法求解而非梯度下降。

TV-L1: Duality Splitting & Iterations 对偶分裂与迭代

Auxiliary field $\mathbf{v}$ decouples the two terms (θ-coupling):

$$\min_{\mathbf{u},\mathbf{v}} \int_\Omega |\nabla u_1| + |\nabla u_2| + \tfrac{1}{2\theta}\,\|\mathbf{u}-\mathbf{v}\|^2 + \lambda\,|\rho(\mathbf{v})|\;d\mathbf{x}$$

1) Pointwise thresholding (closed form in $\mathbf{v}$):

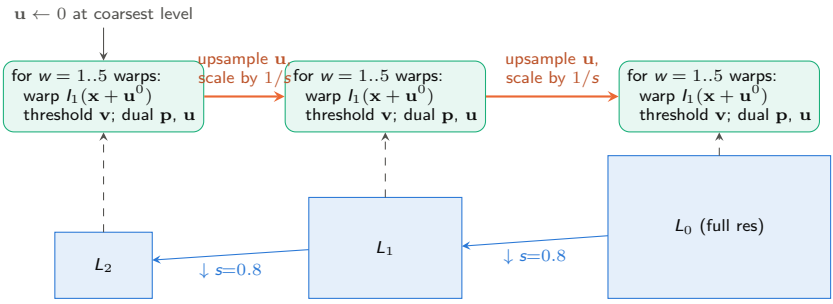
$$\mathbf{v} = \mathbf{u} + \begin{cases} \lambda\theta\,\nabla I_1 & \text{if } \rho(\mathbf{u}) < -\lambda\theta\,|\nabla I_1|^2 \\ -\lambda\theta\,\nabla I_1 & \text{if } \rho(\mathbf{u}) > \lambda\theta\,|\nabla I_1|^2 \\ -\rho(\mathbf{u})\,\nabla I_1/|\nabla I_1|^2 & \text{if } |\rho(\mathbf{u})| \le \lambda\theta\,|\nabla I_1|^2 \end{cases}$$

2) TV proximal step via dual variables $\mathbf{p}_d$ (Chambolle), for $d = 1, 2$:

$$u_d = v_d + \theta\,\mathrm{div}\,\mathbf{p}_d, \qquad \mathbf{p}_d^{n+1} = \frac{\mathbf{p}_d^{\,n} + \frac{\tau}{\theta}\nabla u_d^{n+1}}{1 + \frac{\tau}{\theta}\left|\nabla u_d^{n+1}\right|} \quad \text{(projection onto } |\mathbf{p}_d| \le 1),\ \tau \le 1/4$$

中文要点　引入辅助变量 $\mathbf{v}$ 将两项解耦：第一步对每个像素做闭式阈值化 更新 $\mathbf{v}$；第二步用对偶变量 $\mathbf{p}$ 做 TV 近端步（分母即到单位球的投影）。两步都是纯逐像素运算——非常适合张量化。

Coarse-to-Fine Pyramid + Warping 由粗到细金字塔与图像变形



Linearization ρ is only valid for *small* displacements $\Rightarrow$ estimate coarse motion on small images, then warp I_1 by the upsampled flow and re-linearize at each finer level.

中文要点　线性化残差 ρ 只对小位移成立：先在低分辨率估计大运动，再逐层上采样光流（数值乘 $1/s$），用它对 I_1 做 warp 后重新线性化。每层做 5 次 warp，每次 warp 内做数百次原始-对偶迭代。

DeepFlow: Variational Energy 变分能量

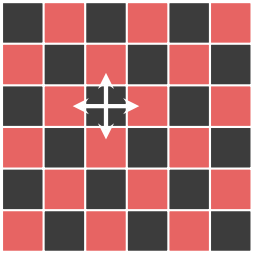
$$
\begin{aligned}
E(\mathbf{w}) \;=\; \int_\Omega \Big[\; & \delta\,\Psi\big(|I_2(\mathbf{x}+\mathbf{w}) - I_1(\mathbf{x})|^2\big) && \text{brightness constancy} \\
& + \gamma\,\Psi\big(\|\nabla I_2(\mathbf{x}+\mathbf{w}) - \nabla I_1(\mathbf{x})\|^2\big) && \text{gradient constancy} \\
& + \alpha\,\Psi\big(\|\nabla u\|^2 + \|\nabla v\|^2\big) \;\Big]\, d\mathbf{x} && \text{smoothness}
\end{aligned}
$$

- Full DeepFlow adds a matching term $\beta\, E_{\text{match}}$ driving the flow toward **DeepMatching** correspondences.
- **OpenCV's `createOptFlow_DeepFlow` implements only the variational part** — no DeepMatching ($\beta = 0$).
- Params (OpenCV defaults): $\alpha = 1$, $\delta = 0.5$, $\gamma = 5$, pyramid downscale 0.95 ($\sim$40+ levels), 5 fixed-point iterations, 25 SOR iterations, $\omega = 1.6$.

中文要点　能量含亮度守恒（权重 δ）、梯度守恒（γ）与平滑项（α），均用鲁棒罚函数 $\Psi(s^2) = \sqrt{s^2 + \varepsilon^2}$。完整 DeepFlow 另有 DeepMatching 匹配项；OpenCV 只实现了变分部分（无匹配项）。

DeepFlow: Fixed-Point + Red-Black SOR 不动点迭代与红黑 SOR

- Euler–Lagrange equations are nonlinear in $\mathbf{w} = (u, v)$: **fixed-point linearization** — freeze Ψ' weights and image derivatives at $\mathbf{w}^k$, solve for the increment $d\mathbf{w}$.
- Each linearization yields a large **sparse linear system** (5-point stencil per pixel, 2 coupled channels).
- Solved by **SOR** ($\omega = 1.6$): each pixel update mixes its own residual with already-updated neighbors.
- **Red-black ordering**: a red pixel's stencil touches only black pixels, so *all red pixels update simultaneously*, then all black — two fully parallel half-sweeps.



A red update reads only **black** neighbors ⇒ each half-sweep is one *vectorized masked update*.

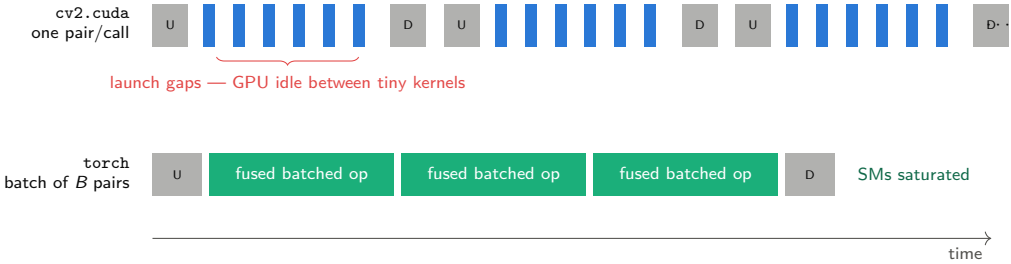
中文要点　外层不动点迭代固定 Ψ' 权重得到稀疏线性方程组，内层用 SOR（$\omega = 1.6$）求解；红黑排序下红格只依赖黑格，红/黑各自可全并行更新——即两次带掩码的张量化更新。

Why OpenCV GPU Is Limited Here OpenCV GPU 路线的局限

1. **Packaging**: pip `opencv-python` wheels ship *without* CUDA ⇒ custom source build per cluster/CUDA version — a real operational burden.
2. **API is one-pair-at-a-time**: `cv2.cuda` takes a single `GpuMat` pair. A 480×640 (let alone 240×320) problem cannot fill an L40's **142 SMs**; upload/download + kernel-launch overhead dominates.
3. **Hundreds of tiny sequential iterations**: TV-L1 ≈ 5 scales × 5 warps × up to 300 primal–dual steps ⇒ thousands of small kernels per pair — *launch-bound*, GPU mostly idle.
4. **DeepFlow: no CUDA version exists in OpenCV at all**; and a naive (lexicographic) SOR sweep is inherently sequential — it must be re-ordered (red-black) to parallelize.

中文要点　（1）pip 版 OpenCV 不含 CUDA，须自行编译；（2）`cv2.cuda` 一次只处理 一对图像，小图无法喂饱 L40 的 142 个 SM，上传/下载与核启动开销占主导；（3）TV-L1 有成千上万个微小串行核，属于启动开销受限；（4）DeepFlow 根本没有 OpenCV CUDA 版本，且朴素 SOR 天生串行。

Launch-Bound vs. Batched 核启动受限 vs. 批量化



Same total math — but one launch now covers B images' worth of pixels, so arithmetic hides the launch latency and fills the GPU.

中文要点　上：cv2.cuda 每对图像触发一串微小核函数，核间空隙里 GPU 空转。 下：批量张量把 B 对图像合成一次大核启动——总计算量相同，但每次启动的 工作量足以摊薄启动延迟并填满所有 SM。

Our Approach: Pure-Torch, Batched 纯 PyTorch 批量化实现

`torch_flow.py` (`calc_flow_tvl1`) · `torch_deepflow.py` (`calc_flow_deepflow`)

- **Batched end to end**: (B, H, W) image pairs $\to$ $(B, 2, H, W)$ flows; every step is a vectorized tensor op over the whole batch — no Python loop over pairs.
- **Warping** = `grid_sample` (bilinear, one call for the batch).
- **Red-black SOR / primal–dual** steps as masked tensor updates: two half-sweeps = two fused elementwise ops.
- **Same code on CPU and CUDA** — just `.to(device)`; no custom OpenCV build.
- **Composable** with a torch pipeline: `DataLoader` feeding, `torch.no_grad()` inference, `torch.compile` for kernel fusion.

中文要点　输入 (B, H, W) 帧对、输出 $(B, 2, H, W)$ 光流，全流程张量化：warp 用 `grid_sample`，红黑 SOR / 原始-对偶迭代化为带掩码的融合逐元素运算。 同一份代码在 CPU 与 GPU 上直接运行，且天然融入 torch 数据管线（DataLoader、`no_grad` 推理、`torch.compile`）。

Runtime Results 运行时间结果

Per-pair wall time at 240×320 (Middlebury), identical algorithm parameters across backends.

Algorithm	cv CPU	cv CUDA	torch CPU	torch CUDA (L40)
TV-L1	424 ms (2.4 FPS)	21.1 ms[a]	8300 ms	**5.5 ms**[b]
DeepFlow	346 ms (2.9 FPS)	— (none)	2263 ms	**16.0 ms**[c] (fp16: 7.4)
Farneback	84 ms @480×640	—	—	—

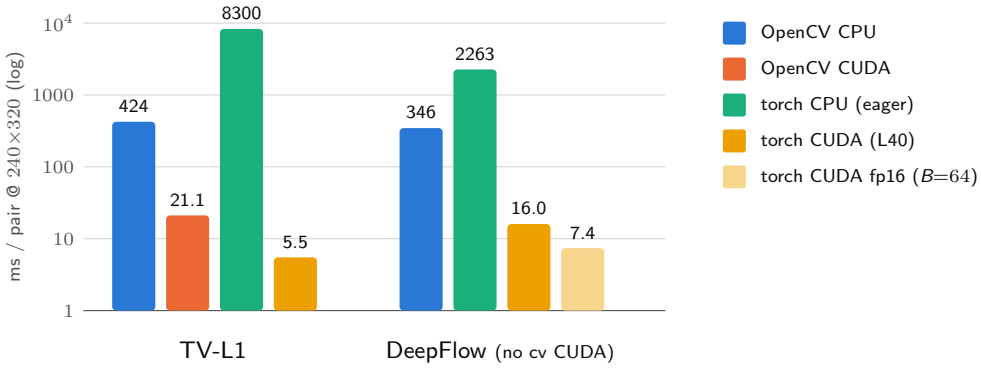
[a] single pair, GPU-resident; 9.3 ms with 4 streams × 4 solver instances; needs custom CUDA 12.9 OpenCV build.

[b] `torch.compile` + adaptive early exit, B=16. [c] `torch.compile`, B=64; per pair = batch time $/B$.

- Multicore doesn't help OpenCV here (Farneback 1 vs 48 threads: 85 vs 84 ms); torch *eager* CPU is even slower — as expected, per-op overhead without fusion.
- Batched torch CUDA: **76**× over OpenCV CPU (TV-L1) and **1.7**× **faster than cv2.cuda's 4-stream ceiling** while computing the *full* algorithm.

中文要点　最终结果：TV-L1 torch CUDA 每对 5.5 ms（compile + 自适应提前退出，B=16），比 OpenCV CPU 快约 76 倍，比 cv2.cuda 四流并发上限（9.3 ms）还快 1.7 倍；DeepFlow 16.0 ms（fp16 为 7.4 ms）。torch eager CPU 反而更慢——符合预期，逐算子开销大且无融合。

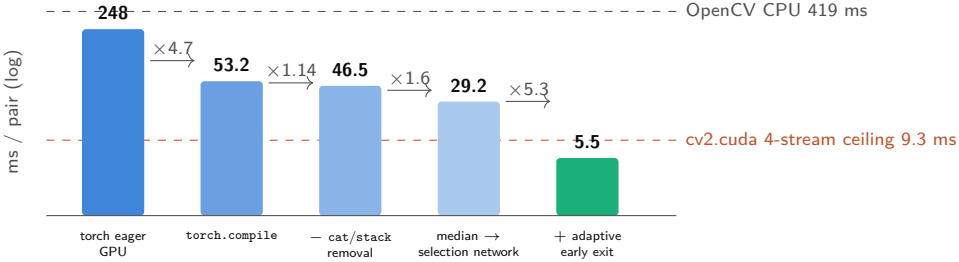
Runtime Chart 运行时间对比图



Log scale — each gridline is $10\times$. torch CUDA spans 3 orders of magnitude below torch eager CPU.

中文要点　对数坐标（每格 10 倍）：torch CUDA（黄）在两种算法上都比 OpenCV CPU 低约 两个数量级；TV-L1 上也低于 cv2.cuda（橙）。torch eager CPU（绿）最慢，印证瓶颈在逐算子开销而非算力。

Optimization Waterfall: TV-L1 on L40 优化阶梯



240×320, B=16: **9.6**× over plain compile, **45**× over eager, **76**× over OpenCV CPU.

- Median filter was 47% of GPU time → replaced by a 202-op min/max **selection network** (18× on that kernel).
- Early exit = OpenCV's own throttled-sync scheme — *bit-for-bit result-neutral*.
- CUDA graphs variant works, but per-step clones make it ∼2× slower than `torch.compile` — compile wins.

中文要点　eager 248 → compile 53.2 → 去 cat/stack 46.5 → 中值滤波改选择网络 29.2 → 自适应提前退出 5.5 ms（与 OpenCV 同款方案，结果逐位不变）；CUDA graphs 因逐步克隆反而慢约 2 倍。

How Strong Is the cv2.cuda Baseline? cv2.cuda 基线有多强

- **Single pair, GPU-resident: 21.1 ms** — but only **42–47% utilization** on the L40: launch-latency-bound even with data resident.
- Best scaling we achieved: **4 CUDA streams** $\times$ **4 solver instances** $\rightarrow$ **9.3 ms/pair** (92% util) — 2.3×, not 4×.
- OpenCV's CUDA TV-L1 is a **lighter variant** of its CPU algorithm (per OpenCV's own parity test): *no median filtering*, real early exit, bicubic $a{=}-0.5$ vs -0.75.
- Our **5.5 ms computes the full CPU algorithm** (median filter included) and still beats the 4-stream ceiling by **1.7**×.
- **DeepFlow**: OpenCV has *no* GPU version at all (`cudaoptflow` = {brox, farneback, nvof, pyrlk, tvl1} only). Our 16.0 ms (fp16: 7.4) vs their CPU 297 ms = **18.6**× (fp16: **40**×) over the only implementation that exists.

中文要点　cv2.cuda 单对 21.1 ms 时利用率仅 42--47%（核启动延迟受限）；四流并发上限 9.3 ms（92% 利用率，2.3 倍而非 4 倍）。且其 CUDA 版是简化变体：无中值滤波、真提前退出、双三次 $a{=}-0.5$。我们的 5.5 ms 计算完整 算法仍快 1.7 倍；DeepFlow 则完全没有 OpenCV GPU 版，16.0 ms（fp16 7.4）对其 CPU 297 ms 为 18.6 倍（fp16 40 倍）。

Scaling with Resolution & GPU Utilization 分辨率扩展与利用率

`torch.compile`, best batch size per resolution (TV-L1 numbers *pre-early-exit*; early exit adds another $\sim$5$\times$ at 240×320).

	TV-L1				DeepFlow			
Resolution	B	ms/pair	util	vs cv CPU	B	ms/pair	util	vs cv CPU
240×320	16	54.5	85%	7.7$\times$	64	16.0	73%	18.6$\times$
480×640	4	224	86%	4.9$\times$	16	71.7	67%	8.8$\times$
480×854	4	323	91%	4.4$\times$	16	99.4	73%	7.5$\times$
720×1280	2	733	92%	4.2$\times$	8	236.7	76%	5.5$\times$

- Saturation batch size falls $\sim 1/\text{pixels}$: once the GPU is full, bigger B stops helping.
- **L2-residency cliff**: working set $\approx 3.7\,\text{MB}\times B$ vs 96 MB L2 predicts the optima (B=16 @240p, B=4 @VGA) *exactly*.
- 99% "utilization" $\neq$ efficient — SMs counted busy while waiting on DRAM.

中文要点　最优批大小随像素数近似反比下降；L2 驻留悬崖：工作集 $\approx 3.7\,\text{MB}\times B$ 对 96 MB L2，恰好预测出 240p 的 B=16 与 VGA 的 B=4。注意高"利用率"不等于高效——SM 在等 DRAM 时也被计为忙。TV-L1 列为未加提前退出的数字（240p 上提前退出再提速约 5 倍）。

Precision Study: fp16, bf16, TF32 数值精度研究

- **fp16 > bf16 for iterative solvers** — mantissa matters (11 vs 8 bits): DeepFlow bf16 $+74\%$ EPE; fp16 only $+0.003$ px.
- fp16 pays off **only at large** B (bandwidth-bound regime): TV-L1 **1.75**×, DeepFlow **2.10**× at $B{=}64$; at small B (launch/latency-bound) it buys nothing.
- **TF32 / autocast are no-ops** here — these solvers contain no matmuls; everything is elementwise/stencil.
- **Keep the 0..255 intensity scale**: rescaling images to 0..1 silently breaks both solvers' tuned thresholds (TV-L1 EPE $0.21 \to 1.46$).

中文要点　迭代求解器更吃尾数位：bf16（8 位尾数）使 DeepFlow EPE 恶化 74%，而 fp16（11 位）仅差 0.003 px。fp16 只在大批量（带宽受限区）才有收益（$B{=}64$：TV-L1 1.75 倍、DeepFlow 2.10 倍）；TF32/autocast 无效（没有矩阵乘）。务必保持 0..255 灰度尺度——归一化到 0..1 会破坏两个 算法的阈值（TV-L1 EPE 从 0.21 恶化到 1.46）。

Accuracy: Endpoint Error 精度：端点误差

Mean EPE on Middlebury pairs at 240×320 (pixels with GT magnitude $> 10^9$, i.e. "unknown", excluded).

Algorithm	Backend	EPE vs. GT	EPE vs. OpenCV ref.
TV-L1	OpenCV CPU	0.208	(reference)
TV-L1	torch	0.208	0.027
DeepFlow	OpenCV CPU	0.186	(reference)
DeepFlow	torch	0.186	0.000*

*$\approx 10^{-6}$ px — the DeepFlow port is **bit-identical** to OpenCV.

- **Full fidelity**: torch matches OpenCV's own GT accuracy exactly (TV-L1: 0.208 both); the 0.027 px TV-L1 gap vs the reference comes from interpolation/boundary choices only.
- DeepFlow's variational part remains the more accurate algorithm (0.186 vs 0.208 EPE) — and our port reproduces it to the bit.

中文要点　精度结论：torch 版与 OpenCV 对真值的 EPE 完全一致（TV-L1 双方均 0.208）；与 OpenCV 参考流场相比，TV-L1 差 0.027 px（仅插值/边界差异），DeepFlow 则逐位一致（差约 10^{-6} px）。DeepFlow（0.186）仍比 TV-L1（0.208）更准。

End-to-End Video Pipeline 端到端视频管线

- Video decode $\rightarrow$ chunked batches $\rightarrow$ flow on GPU: **transfers are** $< 2\%$ of end-to-end runtime — compute-bound, as designed.
- Best chunk size: **16 frames (TV-L1)**, **32 frames (DeepFlow)**.
- Sustained throughput at 240×320 on *one* L40: $\sim$**180 FPS TV-L1** (with early exit), $\sim$**67 FPS DeepFlow** (fp16: **135 FPS**).
- Footprint < 1.7 GB $\Rightarrow$ multiple video streams can share a single GPU.

中文要点　视频解码 $\rightarrow$ 分块成批 $\rightarrow$ GPU 光流：数据搬运占端到端时间不到 2%。 最优块长 TV-L1 为 16 帧、DeepFlow 为 32 帧。单张 L40 在 240×320 下约 180 FPS（TV-L1，含提前退出）、67 FPS（DeepFlow；fp16 达 135 FPS），显存占用低于 1.7 GB，可多路视频共享一卡。

Conclusions & Practical Notes 结论与实用备注

Conclusions

- Batching + `torch.compile` + **algorithm-level porting** (median → selection network, adaptive early exit) **beats even a hand-written CUDA baseline** (5.5 vs 9.3 ms 4-stream ceiling) — at *full* fidelity (DeepFlow bit-identical).
- OpenCV-CUDA's own tricks **transfer cleanly to torch**: kernel fusion via compile, `calcError`-style split, adaptive/throttled sync.
- Red-black reordering makes SOR tensor-friendly; the L2-residency rule picks the batch size per resolution.

Practical notes

- Env: `uv` project; `opencv-contrib-python-headless` on compute nodes (no GUI libs); login node has no GPU — the benchmark script auto-skips unavailable backends.
- GPU runs via SLURM: `srun -p gpu -w ilps-cn119 --gres=gpu:1 --cpus-per-task=8 --mem=16G uv run --no-sync python benchmark.py`

中文要点　批量化 + `torch.compile` + 算法级移植在保持完整保真度的同时 胜过手写 CUDA 基线（5.5 对 9.3 ms）；OpenCV-CUDA 的技巧可干净迁移到 torch。实用：`uv` 管环境，计算节点用 headless OpenCV，GPU 作业经 SLURM 提交至 ilps-cn119。

References 参考文献

1. C. Zach, T. Pock, H. Bischof. *A Duality Based Approach for Realtime TV-L1 Optical Flow.* DAGM Symposium on Pattern Recognition, 2007.
2. P. Weinzaepfel, J. Revaud, Z. Harchaoui, C. Schmid. *DeepFlow: Large Displacement Optical Flow with Deep Matching.* ICCV 2013.
3. J. Sánchez Pérez, E. Meinhardt-Llopis, G. Facciolo. *TV-L1 Optical Flow Estimation.* Image Processing On Line (IPOL), vol. 3, 2013.
4. S. Baker, D. Scharstein, J.P. Lewis, S. Roth, M.J. Black, R. Szeliski. *A Database and Evaluation Methodology for Optical Flow.* IJCV, 2011. (Middlebury benchmark)
5. OpenCV documentation: `cv::optflow::DualTVL1OpticalFlow`, `cv::optflow::createOptFlow_DeepFlow`, `cv::cuda::OpticalFlowDual_TVL1`.

中文要点　以上为 TV-L1（Zach 等 2007；Sánchez 等 IPOL 2013 实现）、DeepFlow（Weinzaepfel 等 ICCV 2013）、Middlebury 数据集及 OpenCV 文档的出处。